

MAI201 MLOps: Assignment 2 - Data Validation & Testing

Monireh eshghinezhad - 06/26/2026

```
In [9]: !pip install -q "numpy>=1.22.4,<2.0.0" "pandas==2.2.2" "great-expectations==0.18.21"
```

```
In [10]: import pandas as pd

df = pd.read_csv("customer_data.csv")
print(df.shape)
print(df.columns.tolist())
print(df.head())
print(df.dtypes)
```

WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).

```
    return datetime.utcnow().replace(tzinfo=utc)
```

```
(5015, 7)
['customer_id', 'age', 'email', 'salary', 'country', 'phone', 'signup_date']
customer_id    age    email    salary    country    phone \
0      C04875  999.0  user4875@example.com  41017.0      USA    3637929158
1      C01026   70.0  user1026@example.com  -1000.0  Australia    -8437
2      C03546   79.0  user3546@example.com   82797.0      USA    423.366.4508
3      C03822   20.0           @domain.com   54624.0      India    719-808-4765
4      C04395   68.0  user4395@example.com  155942.0      USA           NaN
```

```
signup_date
0    9/22/2023
1    2/25/2023
2    9/15/2023
3     8/6/2023
4    5/28/2023
```

```
customer_id    object
age            float64
email          object
salary         float64
country        object
phone          object
signup_date    object
dtype: object
```

Part 1: Great Expectations Setup:

Task: Install Great Expectations and initialize a Great Expectations project in your assignment folder. Configure a data source pointing to the provided CSV file. Create a new expectation suite named `customer_data_expectations`.

```
In [11]: import great_expectations as gx
import os
```

```

# 1. Initialize a Great Expectations project
context = gx.get_context()
print(f"    Context type: {type(context).__name__}")

# 2. Load the CSV and add it as a Data Source
csv_path = "/content/customer_data.csv"

datasource = context.sources.add_pandas_filesystem(
    name="customer_csv_source",
    base_directory=os.path.dirname(csv_path),
)

data_asset = datasource.add_csv_asset(
    name="customer_data_asset",
    batching_regex=r"customer_data\.csv",
)

batch_request = data_asset.build_batch_request()
print("Data source configured--> customer_data.csv")

# 3. Create an Expectation Suite named customer_data_expectations
suite_name = "customer_data_expectations"

suite = context.add_or_update_expectation_suite(
    expectation_suite_name=suite_name
)

print(f"Expectation suite '{suite_name}' created.")
print(f"Suite name: {suite.expectation_suite_name}")

```

```

INFO:great_expectations.data_context.types.base:Created temporary directory '/tmp/tmpnnxpmof4' for ephemeral docs site

```

```

    Context type: EphemeralDataContext
Data source configured--> customer_data.csv
Expectation suite 'customer_data_expectations' created.
Suite name: customer_data_expectations

```

Part 2: Create Expectations

Create the following expectations for the dataset:

- 1.customer_id – must be unique and must not be null.
- 2.age – must be between 0 and 120.
- 3.email – must match a valid email format using regular expressions.
- 4.salary – must be present in at least 95% of rows (use the mostly parameter).
- 5.country – must be one of the following values: USA, Canada, UK, or Australia.
- 6.signup_date – must be of datetime type.
- 7.Overall table row count – must be between 500 and 1000.#

```
In [12]: validator = context.get_validator(  
        batch_request=batch_request,  
        expectation_suite_name=suite_name,  
    )  
    # 1. customer_id: must be unique and must not be null  
    validator.expect_column_values_to_not_be_null(column="customer_id")  
    validator.expect_column_values_to_be_unique(column="customer_id")  
    print("customer_id: not null + unique")  
  
    # 2. age: must be between 0 and 120  
    validator.expect_column_values_to_be_between(  
        column="age",  
        min_value=0,  
        max_value=120,  
    )  
    print("age: between 0 and 120")  
  
    # 3. email: must match valid email format (regex)  
    validator.expect_column_values_to_match_regex(  
        column="email",  
        regex="^[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z]{2,}$",  
    )
```

```

        column="email",
        regex=r"^[a-zA-Z0-9._%+\- ]+@[a-zA-Z0-9.\- ]+\.[a-zA-Z]{2,}$",
    )
    print("email: matches valid email regex")

# 4. salary: must be present in at least 95% of rows
    validator.expect_column_values_to_not_be_null(
        column="salary",
        mostly=0.95,
    )
    print("salary: not null in at least 95% of rows")

# 5. country: must be one of USA, Canada, UK, Australia
    validator.expect_column_values_to_be_in_set(
        column="country",
        value_set=["USA", "Canada", "UK", "Australia"],
    )
    print("country: one of USA, Canada, UK, Australia")

# 6. signup_date: must be parseable as datetime type
    validator.expect_column_values_to_match_strftime_format(
        column="signup_date",
        strftime_format="%m/%d/%Y",
    )
    print("signup_date: (MM/DD/YYYY)")

# 7. Table row count: must be between 500 and 1000
    validator.expect_table_row_count_to_be_between(
        min_value=500,
        max_value=1000,
    )
    print("500 > table row count < 1000")

# Save the expectation suite
    validator.save_expectation_suite(discard_failed_expectations=False)
    print("\n expectations saved to suite:", suite_name)

```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
```

```
    return datetime.utcnow().replace(tzinfo=utc)
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1519: UserWarning: `result_format` configured at the Validator-level will not be persisted. Please add the configuration to your Checkpoint config or checkpoint_run() method instead.
```

```
    warnings.warn(
```

```
Calculating Metrics:  0%|          | 0/6 [00:00<?, ?it/s]
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
```

```
    return datetime.utcnow().replace(tzinfo=utc)
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1519: UserWarning: `result_format` configured at the Validator-level will not be persisted. Please add the configuration to your Checkpoint config or checkpoint_run() method instead.
```

```
    warnings.warn(
```

```
Calculating Metrics:  0%|          | 0/8 [00:00<?, ?it/s]
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
```

```
    return datetime.utcnow().replace(tzinfo=utc)
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1519: UserWarning: `result_format` configured at the Validator-level will not be persisted. Please add the configuration to your Checkpoint config or checkpoint_run() method instead.
```

```
    warnings.warn(
```

```
customer_id: not null + unique
```

Calculating Metrics: 0%| | 0/8 [00:00<?, ?it/s]

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
```

```
    return datetime.utcnow().replace(tzinfo=utc)
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1519: UserWarning: `result_format` configured at the Validator-level will not be persisted. Please add the configuration to your Checkpoint config or checkpoint_run() method instead.
```

```
    warnings.warn(
```

age: between 0 and 120

Calculating Metrics: 0%| | 0/8 [00:00<?, ?it/s]

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
```

```
    return datetime.utcnow().replace(tzinfo=utc)
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
```

```
    return datetime.utcnow().replace(tzinfo=utc)
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1519: UserWarning: `result_format` configured at the Validator-level will not be persisted. Please add the configuration to your Checkpoint config or checkpoint_run() method instead.
```

```
    warnings.warn(
```

email: matches valid email regex

Calculating Metrics: 0%| | 0/6 [00:00<?, ?it/s]

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
```

```
    return datetime.utcnow().replace(tzinfo=utc)
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1519: UserWarning: `result_format` configured at the Validator-level will not be persisted. Please add the configuration to your Checkpoint config or checkpoint_run() method instead.
```

```
    warnings.warn(
```

```
salary: not null in at least 95% of rows
```

```
Calculating Metrics:  0%|          | 0/8 [00:00<?, ?it/s]
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
```

```
    return datetime.utcnow().replace(tzinfo=utc)
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1519: UserWarning: `result_format` configured at the Validator-level will not be persisted. Please add the configuration to your Checkpoint config or checkpoint_run() method instead.
```

```
    warnings.warn(
```

```
country: one of USA, Canada, UK, Australia
```

```
Calculating Metrics:  0%|          | 0/8 [00:00<?, ?it/s]
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
```

```
    return datetime.utcnow().replace(tzinfo=utc)
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1519: UserWarning: `result_format` configured at the Validator-level will not be persisted. Please add the configuration to your Checkpoint config or checkpoint_run() method instead.
```

```
    warnings.warn(
```

```
signup_date:(MM/DD/YYYY)
```

```
Calculating Metrics:  0%|          | 0/1 [00:00<?, ?it/s]
```

```
500 > table row count < 1000
```

```
expectations saved to suite: customer_data_expectations
```

Part 3: Data Quality Report

Run the expectation suite against the messy dataset and capture the validation results. Generate and save the Great Expectations data documentation as an HTML file. Identify and document all data quality issues found, including counts per issue.

```
In [13]: import json
```

```
# 1. Run the validation checkpoint
```

```
validation_result = validator.validate()
```

```
print("=" * 60)
```

```
print("VALIDATION SUMMARY")
```

```
print("=" * 60)
```

```
print(f"Success:      {validation_result['success']}")
```

```
print(f"Total expectations: {validation_result['statistics']['evaluated_expectations']}")
```

```

print(f"Passed:      {validation_result['statistics']['successful_expectations']}")
print(f"Failed:      {validation_result['statistics']['unsuccessful_expectations']}")
print(f"Success rate:{validation_result['statistics']['success_percent']:.1f}%")

# 2. Generate HTML Data Docs and save to file
# Build data docs (GE generates HTML automatically)
context.build_data_docs()

# Find the generated HTML file
docs_site_path = None
for site_name, site_config in context.get_config().data_docs_sites.items():
    base_dir = site_config.get("store_backend", {}).get("base_directory", None)
    if base_dir:
        docs_site_path = base_dir
        break

# HTML report
html_report_path = "/content/data_quality_report.html"

html_content = f"""
<html>
<head><title>Data Quality Report - customer_data</title>
<style>
    body {{ font-family: Arial, sans-serif; margin: 30px; }}
    h1 {{ color: #2c3e50; }}
    h2 {{ color: #34495e; border-bottom: 2px solid #3498db; padding-bottom: 5px; }}
    table {{ border-collapse: collapse; width: 100%; margin-bottom: 20px; }}
    th {{ background-color: #3498db; color: white; padding: 10px; text-align: left; }}
    td {{ border: 1px solid #ddd; padding: 8px; }}
    tr:nth-child(even) {{ background-color: #f2f2f2; }}
    .pass {{ color: green; font-weight: bold; }}
    .fail {{ color: red; font-weight: bold; }}
    .summary-box {{ background: #ecf0f1; padding: 15px; border-radius: 8px; margin-bottom: 20px; }}
</style>
</head>
<body>

```

```

<h1><img alt="Data Quality Report icon" data-bbox="165 5 185 25"/> Data Quality Report – customer_data.csv</h1>
<div class="summary-box">
    <strong>Overall Result:</strong> {' ** PASSED **' if validation_result['success'] else ' ** FA
    <strong>Total Expectations:</strong> {validation_result['statistics']['evaluated_expectations']
    <strong>Passed:</strong> {validation_result['statistics']['successful_expectations']}<br>
    <strong>Failed:</strong> {validation_result['statistics']['unsuccessful_expectations']}<br>
    <strong>Success Rate:</strong> {validation_result['statistics']['success_percent']:.1f}%
</div>
<h2>Expectation Results</h2>
<table>
    <tr><th>Column</th><th>Expectation</th><th>Result</th><th>Failing Count</th></tr>
    ""

for result in validation_result["results"]:
    col = result["expectation_config"].get("kwargs", {}).get("column", "TABLE")
    exp_type = result["expectation_config"]["expectation_type"]
    success = result["success"]
    status = '<span class="pass"> ** PASS **</span>' if success else '<span class="fail"> ** FAI

    # Get unexpected count if available
    unexpected_count = result.get("result", {}).get("unexpected_count", "N/A")
    if success:
        unexpected_count = 0

    html_content += f"    <tr><td>{col}</td><td>{exp_type}</td><td>{status}</td><td>{unexpected_co

html_content += "</table></body></html>"

with open(html_report_path, "w") as f:
    f.write(html_content)

print(f"\n HTML report saved to: {html_report_path}")

# 3. Document all data quality issues with counts
print("\n" + "=" * 60)
print("DATA QUALITY ISSUES FOUND")

```

```

print("=" * 60)

issue_count = 0
for result in validation_result["results"]:
    if not result["success"]:
        issue_count += 1
        col = result["expectation_config"].get("kwargs", {}).get("column", "TABLE-LEVEL")
        exp_type = result["expectation_config"]["expectation_type"]
        res = result.get("result", {})

        unexpected_count = res.get("unexpected_count", "N/A")
        unexpected_pct = res.get("unexpected_percent", None)
        element_count = res.get("element_count", "N/A")
        unexpected_values = res.get("partial_unexpected_list", [])

        print(f"\n Issue #{issue_count}")
        print(f"    Column      : {col}")
        print(f"    Expectation : {exp_type}")
        print(f"    Failing rows: {unexpected_count} / {element_count}", end="")
        if unexpected_pct is not None:
            print(f"    ({unexpected_pct:.2f}%)")
        else:
            print()
        if unexpected_values:
            print(f"    Sample bad values: {unexpected_values[:5]}")

if issue_count == 0:
    print(" No issues found – all expectations passed!")
else:
    print(f"\n{'=' * 60}")
    print(f"Total issues: {issue_count} expectation(s) failed")

# 4. Summary table using pandas
print("\n" + "=" * 60)
print("SUMMARY TABLE")
print("=" * 60)

```

```

rows = []
for result in validation_result["results"]:
    col = result["expectation_config"].get("kwargs", {}).get("column", "TABLE")
    exp_type = result["expectation_config"]["expectation_type"]
    success = result["success"]
    res = result.get("result", {})
    rows.append({
        "Column" : col,
        "Expectation" : exp_type,
        "Passed" : "Yes " if success else "No",
        "Failing Rows" : res.get("unexpected_count", 0) if not success else 0,
        "Failing %" : f"{res.get('unexpected_percent', 0):.2f}%" if not success else "0.00%"
    })

summary_df = pd.DataFrame(rows)
print(summary_df.to_string(index=False))

```

Calculating Metrics: 0% | 0/32 [00:00<?, ?it/s]

WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).

```
return datetime.utcnow().replace(tzinfo=utc)
```

WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).

```
return datetime.utcnow().replace(tzinfo=utc)
```

```
=====
VALIDATION SUMMARY
=====
```

```
Success:      False
Total expectations: 8
Passed:       0
Failed:       8
Success rate:0.0%
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)
```

```
=====
DATA QUALITY ISSUES FOUND
=====
```

Issue #1

Column : customer_id
Expectation : expect_column_values_to_not_be_null
Failing rows: 150 / 5015 (2.99%)
Sample bad values: [nan, nan, nan, nan, nan]

Issue #2

Column : customer_id
Expectation : expect_column_values_to_be_unique
Failing rows: 568 / 5015 (11.68%)
Sample bad values: ['C00923', 'C01991', 'C00380', 'C00801', 'C00113']

Issue #3

Column : age
Expectation : expect_column_values_to_be_between
Failing rows: 384 / 5015 (7.89%)
Sample bad values: [999.0, -25.0, 500.0, -5.0, 300.0]

Issue #4

Column : email
Expectation : expect_column_values_to_match_regex
Failing rows: 346 / 5015 (7.56%)
Sample bad values: ['@domain.com', 'invalid-email', 'user@@domain.com', 'no-dot@com', 'user@']

Issue #5

Column : salary
Expectation : expect_column_values_to_not_be_null
Failing rows: 425 / 5015 (8.47%)
Sample bad values: [nan, nan, nan, nan, nan]

Issue #6
 Column : country
 Expectation : expect_column_values_to_be_in_set
 Failing rows: 301 / 5015 (6.05%)
 Sample bad values: ['India', 'Spain', 'Brazil', 'Brazil', 'Brazil']

Issue #7
 Column : signup_date
 Expectation : expect_column_values_to_match_strftime_format
 Failing rows: 242 / 5015 (4.84%)
 Sample bad values: ['2023-02-30', '2023-02-29', '2023-04-31', '2023-02-30', '2023-04-31']

Issue #8
 Column : TABLE-LEVEL
 Expectation : expect_table_row_count_to_be_between
 Failing rows: N/A / N/A

=====
 Total issues: 8 expectation(s) failed

=====
 SUMMARY TABLE

=====					
Column	Expectation	Passed	Failing Rows	Failing %	
customer_id	expect_column_values_to_not_be_null	No	150	2.99%	
customer_id	expect_column_values_to_be_unique	No	568	11.68%	
age	expect_column_values_to_be_between	No	384	7.89%	
email	expect_column_values_to_match_regex	No	346	7.56%	
salary	expect_column_values_to_not_be_null	No	425	8.47%	
country	expect_column_values_to_be_in_set	No	301	6.05%	
signup_date	expect_column_values_to_match_strftime_format	No	242	4.84%	
TABLE	expect_table_row_count_to_be_between	No	0	0.00%	

Part 4: Write pytest Unit Tests

Write pytest unit tests for three data utility functions:

- 1.load_csv(filepath) – test for file not found, empty file, and successful loading.
- 2.clean_phone(phone) – test various input formats and invalid inputs to ensure consistent output.
- 3.validate_email(email) – test valid emails, invalid emails, and edge cases.

```
In [14]: !pip install ipytest -q
```

```
In [15]: import re

# Function 1: load_csv
def load_csv(filepath):
    """Load a CSV file and return a DataFrame."""
    if not os.path.exists(filepath):
        raise FileNotFoundError(f"File not found: {filepath}")

    df = pd.read_csv(filepath)

    if df.empty:
        raise ValueError(f"File is empty: {filepath}")

    return df

# Function 2: clean_phone number
def clean_phone(phone):
    """
    Clean a phone number by removing all non-digit characters.
    Returns digits only as a string, or None if invalid.
    """
    if phone is None or (isinstance(phone, float) and pd.isna(phone)):
        return None

    phone = str(phone)
    digits = re.sub(r'\D', '', phone) # remove everything except digits
```

```

    if len(digits) < 7 or len(digits) > 15:
        return None # too short or too long to be a valid phone number

    return digits

# Function 3: validate_email
def validate_email(email):
    """
    Validate an email address using regex.
    Returns True if valid, False otherwise.
    """
    if not email or not isinstance(email, str):
        return False

    pattern = r'^[a-zA-Z0-9._%+\-]+\@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$'
    return bool(re.match(pattern, email.strip()))

print("Utility functions defined: load_csv, clean_phone, validate_email")

```

Utility functions defined: load_csv, clean_phone, validate_email

```

In [16]: # pytest Unit Tests
import ipytest
import pytest
import tempfile

ipytest.autoconfig()

# -----
# Tests for load_csv()

def test_load_csv_file_not_found():
    """load_csv raises FileNotFoundError for a non-existent file."""
    with pytest.raises(FileNotFoundError):

```

```
load_csv("/content/nonexistent_file.csv")
```

```
def test_load_csv_empty_file():
    """load_csv raises ValueError when the CSV file is empty."""
    with tempfile.NamedTemporaryFile(mode='w', suffix='.csv',
                                      delete=False) as f:
        f.write("")          # write nothing – empty file
        tmp_path = f.name
    try:
        with pytest.raises(ValueError):
            load_csv(tmp_path)
    finally:
        os.unlink(tmp_path)

def test_load_csv_successful():
    """load_csv returns a non-empty DataFrame on a valid CSV file."""
    with tempfile.NamedTemporaryFile(mode='w', suffix='.csv',
                                      delete=False) as f:
        f.write("name,age\nAlice,30\nBob,25\n")
        tmp_path = f.name
    try:
        df = load_csv(tmp_path)
        assert df is not None
        assert len(df) == 2
        assert list(df.columns) == ["name", "age"]
    finally:
        os.unlink(tmp_path)

def test_load_csv_returns_dataframe():
    """load_csv returns a pandas DataFrame type."""
    with tempfile.NamedTemporaryFile(mode='w', suffix='.csv',
                                      delete=False) as f:
        f.write("id,value\n1,100\n")
```

```

        tmp_path = f.name
    try:
        result = load_csv(tmp_path)
        assert isinstance(result, pd.DataFrame)
    finally:
        os.unlink(tmp_path)

# -----
# Tests for clean_phone()

def test_clean_phone_standard_format():
    """clean_phone removes dashes from a standard phone number."""
    assert clean_phone("719-808-4765") == "7198084765"

def test_clean_phone_with_dots():
    """clean_phone removes dots from a dotted phone number."""
    assert clean_phone("423.366.4508") == "4233664508"

def test_clean_phone_with_spaces():
    """clean_phone removes spaces from a phone number."""
    assert clean_phone("719 808 4765") == "7198084765"

def test_clean_phone_with_parentheses():
    """clean_phone removes parentheses and dashes."""
    assert clean_phone("(719) 808-4765") == "7198084765"

def test_clean_phone_plain_digits():
    """clean_phone returns plain digits unchanged."""
    assert clean_phone("3637929158") == "3637929158"

def test_clean_phone_none_input():

```

```

"""clean_phone returns None for None input."""
assert clean_phone(None) is None

def test_clean_phone_nan_input():
    """clean_phone returns None for NaN input."""
    import math
    assert clean_phone(float('nan')) is None

def test_clean_phone_too_short():
    """clean_phone returns None for a number that is too short."""
    assert clean_phone("123") is None

def test_clean_phone_negative_number():
    """clean_phone returns None for an invalid negative number."""
    assert clean_phone("-8437") is None

# -----
# Tests for validate_email()

def test_validate_email_valid():
    """validate_email returns True for a standard valid email."""
    assert validate_email("user@example.com") is True

def test_validate_email_valid_with_subdomain():
    """validate_email returns True for email with subdomain."""
    assert validate_email("user@mail.example.co.uk") is True

def test_validate_email_valid_with_plus():
    """validate_email returns True for email with plus sign."""
    assert validate_email("user+tag@example.com") is True

```

```
def test_validate_email_missing_at_symbol():
    """validate_email returns False when @ is missing."""
    assert validate_email("userdomain.com") is False

def test_validate_email_missing_domain():
    """validate_email returns False when domain is missing."""
    assert validate_email("user@") is False

def test_validate_email_starts_with_at():
    """validate_email returns False when email starts with @."""
    assert validate_email("@domain.com") is False

def test_validate_email_missing_tld():
    """validate_email returns False when TLD is missing."""
    assert validate_email("user@domain") is False

def test_validate_email_empty_string():
    """validate_email returns False for empty string."""
    assert validate_email("") is False

def test_validate_email_none():
    """validate_email returns False for None input."""
    assert validate_email(None) is False

def test_validate_email_spaces():
    """validate_email returns False for email with spaces."""
    assert validate_email("user name@domain.com") is False
```

```
# Run all tests
ipytest.run('-v')
```

```
===== test session starts =====
===
platform linux -- Python 3.12.13, pytest-8.4.2, pluggy-1.6.0
rootdir: /content
plugins: langsmith-0.8.15, typeguard-4.5.2, anyio-4.13.0
collected 23 items

t_d413e6fc0bd046728d5d4db21c444cea.py .....

WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)

[100%]

WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)

===== warnings summary =====
===
t_d413e6fc0bd046728d5d4db21c444cea.py: 23 warnings
  /usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 23 passed, 23 warnings in 0.06s =====
===
```

```
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).  
    return datetime.utcnow().replace(tzinfo=utc)
```

Out[16]: <ExitCode.OK: 0>